

# Diccionario de Términos Técnicos

Referencia completa de conceptos de finanzas, programación, análisis de datos y herramientas

Sebastián Egaña Santibáñez (segana@fen.uchile.cl)  
Nicolás Leiva Díaz (nleivad@fen.uchile.cl)

2026-09-28

## Descargas y Enlaces

---

# Diccionario de términos técnicos, FAQ y recursos

Esta sección reúne conceptos de **finanzas, programación, análisis de datos y herramientas tecnológicas** utilizados durante las clases de Finanzas en R.

El objetivo es disponer de una referencia rápida para revisar los notebooks, ejercicios y código desarrollado durante el curso.

---

# Conceptos de finanzas y análisis cuantitativo

## Acción

Instrumento financiero que representa una participación en una empresa.

En los ejercicios se utilizaron como ejemplo acciones identificadas mediante tickers como AAPL, GOOGL y MSFT.

---

## Activo

Instrumento o recurso financiero susceptible de generar retornos.

Una acción es un ejemplo de activo financiero.

---

## Ticker

Código utilizado para identificar un activo financiero en los mercados.

Ejemplos:

- AAPL: Apple.
  - GOOGL: Alphabet.
  - MSFT: Microsoft.
- 

## Retorno

Variación porcentual del valor de un activo durante un período.

En los ejercicios se simulaban retornos mediante:

```
rnorm(  
  n = n_dias,  
  mean = 0.0005,
```

```
sd = 0.015  
)
```

En este caso se utiliza una media diaria de **0,05 %** y una desviación estándar de **1,5 %**.

---

## Retorno diario

Retorno de un activo medido entre un día y el siguiente.

En distintos ejercicios se utilizaron **252 observaciones** como aproximación a un año bursátil.

---

## Retorno esperado

Valor promedio que se espera obtener para un activo o portafolio.

En las simulaciones iniciales:

```
media <- 0.0005
```

representa un retorno esperado diario de aproximadamente **0,05 %**.

En el ejercicio de portafolios, los retornos esperados de cada activo se calcularon mediante:

```
retornos_esperados <- colMeans(retornos_matrix)
```

---

## Retorno acumulado

Representa el rendimiento obtenido al reinvertir sucesivamente los retornos de cada período.

En R:

```
retorno_acumulado = cumprod(1 + retorno) - 1
```

La fórmula general corresponde a:

```
(1+r1)(1+r2)...(1+rn)-1
```

Por esta razón, los retornos porcentuales de distintos períodos no deben simplemente sumarse cuando se desea obtener el rendimiento compuesto.

---

## Retorno objetivo

Nivel de retorno que se busca alcanzar al construir un portafolio.

En el ejercicio de Markowitz se analizaron distintos retornos objetivo:

```
retornos_objetivo <- seq(  
  0.08,  
  0.20,  
  length.out = 50  
) / 12
```

Esto genera 50 niveles de retorno entre 8 % y 20 % anual, transformados a valores mensuales para mantener consistencia con la frecuencia de los datos utilizados.

---

## Desviación estándar

Medida estadística que representa cuánto se dispersan los valores respecto de su media.

En los ejercicios:

```
desv_est <- 0.015
```

corresponde a una desviación estándar diaria de **1,5 %**.

En finanzas se utiliza frecuentemente como medida de volatilidad.

---

## Volatilidad

Medida de la variabilidad de los retornos de un activo o portafolio.

Una mayor volatilidad indica que los retornos presentan mayores fluctuaciones alrededor de su promedio.

En el ejercicio de portafolios se calculó la volatilidad mediante la matriz de covarianzas y los pesos del portafolio:

```
vol_opt <- sqrt(  
  t(w_opt) %*%  
  cov_matrix %*%  
  w_opt  
) * sqrt(12)
```

El factor `sqrt(12)` anualiza una volatilidad calculada con datos mensuales.

---

## Varianza

Medida estadística que cuantifica la dispersión de una variable respecto de su media.

La desviación estándar corresponde a la raíz cuadrada de la varianza.

En teoría de portafolios, la varianza permite representar matemáticamente el riesgo del portafolio.

---

## Covarianza

Medida de cómo dos variables tienden a cambiar conjuntamente.

En finanzas se utiliza para estudiar cómo se relacionan los retornos de distintos activos.

Una covarianza positiva indica que ambos activos tienden a moverse en la misma dirección, mientras que una covarianza negativa indica una tendencia a moverse en direcciones opuestas.

---

## Matriz de covarianzas

Matriz que contiene las varianzas y covarianzas entre todos los activos de un conjunto.

En clase se calculó mediante:

```
cov_matrix <- cov(retornos_matrix)
```

Esta matriz es uno de los elementos fundamentales de la optimización de portafolios de Markowitz.

---

## Correlación

Medida estadística estandarizada del grado de asociación lineal entre dos variables.

Para retornos financieros permite estudiar en qué medida dos activos tienden a moverse conjuntamente.

Una correlación cercana a:

- 1: indica movimientos lineales similares.
- 0: indica poca relación lineal.
- -1: indica movimientos lineales en direcciones opuestas.

La correlación no implica causalidad.

---

## Matriz de correlación

Tabla que contiene las correlaciones entre varios activos.

En clase:

```
tabla_correlacion <- datos %>%  
  pivot_wider(  
    names_from = ticker,  
    values_from = retorno  
  ) %>%
```

```
select(-fecha) %>%  
cor(use = "complete.obs")
```

---

# Teoría de portafolios

## Portafolio

Conjunto de activos financieros mantenidos simultáneamente por un inversionista.

Cada activo representa una determinada proporción de la inversión total.

---

## Peso de un activo

Proporción del capital del portafolio invertida en un determinado activo.

Se representa normalmente mediante:

```
w
```

Si un portafolio tiene tres activos, podría tener pesos como:

```
Activo A = 40 %  
Activo B = 35 %  
Activo C = 25 %
```

La suma de los pesos sería:

```
40 % + 35 % + 25 % = 100 %
```

---

## Restricción presupuestaria

Condición que establece que la totalidad del capital debe distribuirse entre los activos.

En el código:

```
sum(w) = 1
```

y se incorpora mediante:

```
rep(1, n_activos)
```

---

## Venta corta o Short Selling

Operación en la cual se toma una posición negativa sobre un activo.

En el ejercicio realizado en clase se incorporó la restricción:

$$w \geq 0$$

lo que implica que **no se permiten pesos negativos** y, por lo tanto, el problema planteado no contempla posiciones cortas.

---

## Diversificación

Distribución de una inversión entre distintos activos.

La idea central es que el riesgo de un portafolio depende no solamente del riesgo individual de sus activos, sino también de cómo estos activos se relacionan entre sí.

Por esto la matriz de covarianzas resulta fundamental.

---

## Modelo de Markowitz

Modelo de selección de portafolios que analiza simultáneamente:

- retorno esperado;
- riesgo;
- pesos de los activos;
- correlaciones o covarianzas entre activos.

El problema puede formularse buscando el portafolio de **mínima varianza para un determinado retorno objetivo**.

---

## Optimización Media-Varianza

Enfoque utilizado por el modelo de Markowitz.

Considera dos dimensiones principales:

|          |                    |
|----------|--------------------|
| Media    | → retorno esperado |
| Varianza | → riesgo           |

El objetivo es encontrar combinaciones de activos que entreguen determinadas relaciones entre retorno y riesgo.

---

## Frontera eficiente

Conjunto de portafolios que representan combinaciones eficientes entre riesgo y retorno.

Para cada nivel de retorno objetivo se busca el portafolio con menor riesgo posible.

En clase se creó una estructura llamada:

```
frontera_markowitz
```

que almacena:

```
frontera_markowitz <- list(  
  retorno = ...,  
  volatilidad = ...,  
  sharpe = ...,  
  pesos = ...  
)
```

Cada punto de la frontera representa un portafolio diferente.

---

## Programación cuadrática

Método matemático de optimización en el cual la función objetivo contiene términos cuadráticos.

La minimización de la varianza de un portafolio puede expresarse como un problema de programación cuadrática.

En R se utilizó el paquete:

```
library(quadprog)
```

y la función:

```
solve.QP()
```

---

## Restricciones del problema de optimización

En el ejercicio de clase se utilizaron tres grupos principales de condiciones:

1. Los pesos deben sumar 1.
2. El portafolio debe alcanzar el retorno objetivo.
3. Los pesos deben ser mayores o iguales a 0.

En el código:

```
A <- cbind(  
  rep(1, n_activos),
```

```
retornos_esperados,  
diag(n_activos)  
)
```

---

## Tasa libre de riesgo

Retorno teórico de una inversión considerada sin riesgo para efectos del modelo.

En el ejercicio se utilizó:

```
tasa_libre_riesgo <- 0.02 / 12
```

correspondiente a una tasa anual de **2%**, transformada a una aproximación mensual mediante división por 12.

---

## Ratio de Sharpe

Indicador que relaciona el retorno adicional obtenido respecto de la tasa libre de riesgo con la volatilidad asumida.

Su estructura general es:

```
Sharpe =  
(Retorno del portafolio - Tasa libre de riesgo)  
/  
Volatilidad del portafolio
```

En clase:

```
frontera_markowitz$sharpe <- ifelse(  
  frontera_markowitz$volatilidad > 1e-10,  
  (frontera_markowitz$retorno - 0.02) /  
    frontera_markowitz$volatilidad,  
  NA  
)
```

Un mayor Sharpe representa una mayor cantidad de retorno excedente por unidad de volatilidad dentro de este criterio.

---

## Portafolio tangente

Portafolio de la frontera eficiente que maximiza el ratio de Sharpe respecto de una determinada tasa libre de riesgo.

En el ejercicio se identificó utilizando:

```
idx_tangente <-  
  which.max(frontera_markowitz$sharpe)
```

Posteriormente se almacenaron sus características:

```
portafolio_tangente <- list(  
  retorno = ...,  
  volatilidad = ...,  
  sharpe = ...,  
  pesos = ...  
)
```

---

## Capital Allocation Line — CAL

La **Capital Allocation Line** representa combinaciones entre:

- el activo libre de riesgo;
- un portafolio riesgoso.

En el gráfico desarrollado durante la clase se muestra junto a la frontera eficiente:

```
labs(  
  title = "Frontera Eficiente y Capital Allocation Line"  
)
```

El portafolio tangente corresponde al punto donde la CAL es tangente a la frontera eficiente bajo los supuestos del modelo.

---

## Anualización

Transformación de una métrica calculada con frecuencia diaria o mensual a una escala anual.

Por ejemplo, en el código los retornos mensuales se transforman nuevamente a escala anual mediante:

```
r_obj * 12
```

Mientras que la volatilidad mensual se anualiza mediante:

```
volatilidad_mensual * sqrt(12)
```

Es importante distinguir ambas transformaciones: retorno y volatilidad no se anualizan exactamente de la misma manera en este ejercicio.

# Simulación y probabilidad

## Distribución normal

Distribución de probabilidad utilizada en los ejercicios para simular retornos mediante:

```
rnorm()
```

Se caracteriza principalmente por una media y una desviación estándar.

---

## Distribución triangular

Distribución de probabilidad definida mediante tres parámetros:

- mínimo;
- máximo;
- moda o valor más probable.

En clase:

```
rtriangle(1, 9, 13, 11)
```

representa:

```
mínimo = 9  
máximo = 13  
más probable = 11
```

---

## Simulación

Proceso mediante el cual se generan valores artificiales utilizando determinados supuestos.

Ejemplo:

```
retorno = rnorm(  
  n(),  
  mean = 0.0005,  
  sd = 0.015  
)
```

---

## Simulación Monte Carlo

Método que repite múltiples veces un experimento que contiene variables aleatorias para estudiar los distintos resultados posibles.

En uno de los ejercicios:

```
reps = 10000
```

indica que la simulación se ejecuta 10.000 veces.

---

## Escenario

Una posible combinación de condiciones dentro de una simulación.

En clase se trabajó con:

```
sample(  
  c("A", "B", "C"),  
  1,  
  replace = TRUE,  
  prob = c(1/4, 1/2, 1/4)  
)
```

---

## Probabilidad

Medida de qué tan probable es que ocurra determinado resultado.

En una simulación permite asignar distintos pesos a posibles escenarios.

---

# Conceptos de programación y análisis de datos

## R

Lenguaje de programación especialmente utilizado en estadística, ciencia de datos, visualización y modelamiento.

Es el lenguaje principal del curso.

---

## Python

Lenguaje de programación de propósito general muy utilizado en ciencia de datos, automatización e inteligencia artificial.

---

## RStudio

Entorno de desarrollo especializado tradicionalmente en R.

Permite trabajar con scripts, consola, gráficos, paquetes, archivos y proyectos.

---

## Positron

Entorno de desarrollo orientado a ciencia de datos que permite trabajar con lenguajes como R y Python.

Positron y RStudio son aplicaciones independientes.

---

## IDE

**Integrated Development Environment.**

Aplicación que reúne herramientas para escribir, ejecutar, revisar y depurar código.  
RStudio y Positron son ejemplos.

---

## Script

Archivo que contiene una secuencia de instrucciones de programación.

Un script puede, por ejemplo:

```
Cargar datos  
→ Transformarlos  
→ Calcular indicadores  
→ Generar gráficos  
→ Exportar resultados
```

---

## Objeto

Elemento almacenado en memoria dentro de R.

```
x <- 2
```

crea un objeto llamado x.

---

## Asignación <-

Operador utilizado habitualmente en R para guardar información dentro de un objeto.

```
media <- 0.0005
```

---

## Función

Bloque de código diseñado para realizar una tarea.

Ejemplos utilizados durante las clases:

```
rnorm()  
mean()  
cov()  
cor()  
ggplot()  
read_csv()  
solve.QP()
```

---

## Paquete

Colección de funciones que amplían las capacidades de R.

Entre los paquetes utilizados durante las clases se encuentran:

```
tidyverse  
ggplot2  
dplyr  
readr  
lubridate  
triangle  
scales  
quadprog
```

---

## library()

Carga un paquete instalado.

```
library(dplyr)
```

---

## install.packages()

Instala un paquete.

```
install.packages("rmarkdown")
```

---

# Manipulación y estructuras de datos

## tidyverse

Ecosistema de paquetes de R diseñado para análisis y transformación de datos.

Incluye herramientas como:

```
dplyr  
ggplot2  
readr  
tidyr  
tibble
```

---

## dplyr

Paquete utilizado para transformar y manipular datos.

Entre las funciones trabajadas:

```
select()  
mutate()  
arrange()  
group_by()  
ungroup()
```

---

## Pipe %>%

Operador utilizado tradicionalmente dentro del ecosistema tidyverse para encadenar operaciones.

```
datos %>%  
  group_by(ticker) %>%  
  mutate(...)
```

---

## Pipe nativo |>

Operador incorporado directamente en versiones modernas de R.

Durante la clase de portafolios se utilizó:

```
portafolio_retornos |>
  select(-fecha) |>
  as.matrix()
```

Su objetivo es similar al de %>%: facilitar la lectura de una secuencia de operaciones.

---

## Data frame

Estructura tabular utilizada en R para almacenar datos organizados en filas y columnas.

---

## Tibble

Versión moderna de un `data.frame`, ampliamente utilizada por tidyverse.

Durante el ejercicio de Markowitz:

```
portafolio_tangente_df <- tibble(
  retorno = ...,
  volatilidad = ...,
  sharpe = ...,
  pesos = ...
)
```

## Matriz

Estructura bidimensional en la cual todos los elementos son del mismo tipo.

En el ejercicio de portafolios:

```
retornos_matrix <-
  portafolio_retornos |>
  select(-fecha) |>
  as.matrix()
```

## as.matrix()

Convierte un objeto compatible en una matriz.

---

## **colMeans()**

Calcula la media de cada columna de una matriz o estructura similar.

```
retornos_esperados <-  
  colMeans(retornos_matrix)
```

---

## **cov()**

Calcula covarianzas.

```
cov_matrix <-  
  cov(retornos_matrix)
```

---

## **ncol()**

Entrega el número de columnas de una estructura.

```
n_activos <-  
  ncol(retornos_matrix)
```

En el ejercicio corresponde al número de activos del portafolio.

---

## **list()**

Crea una estructura capaz de almacenar objetos de distintos tipos.

En clase se utilizó para almacenar simultáneamente:

```
retorno  
volatilidad  
Sharpe  
pesos
```

---

# Datos y archivos

## CSV

### Comma-Separated Values.

Formato de texto utilizado para intercambiar datos tabulares.

Durante las clases se utilizaron archivos como:

```
retornos.csv  
retornos_3_acciones.csv  
portafolio_retornos.csv
```

---

### **read\_csv()**

Función de `readr` utilizada para importar CSV.

```
portafolio_retornos <-  
  read_csv(  
    "data/portafolio_retornos.csv"  
  )
```

---

### **write\_csv()**

Exporta información a CSV.

```
write_csv(  
  datos_simulados,  
  "retornos_3_acciones.csv"  
)
```

---

## Ruta relativa

Ubicación de un archivo expresada respecto de la carpeta del proyecto.

Por ejemplo:

```
data/portafolio_retornos.csv
```

indica que existe una carpeta llamada **data** dentro del proyecto.

Utilizar rutas relativas facilita la reproducibilidad de un proyecto.

---

# Visualización de datos

## ggplot2

Paquete utilizado para construir gráficos en R mediante una gramática de visualización.

---

## aes()

Define la relación entre variables y propiedades visuales.

```
aes(  
  x = volatilidad,  
  y = retorno  
)
```

---

## geom\_line()

Genera líneas.

Se utilizó tanto para retornos acumulados como para representar la frontera eficiente.

---

## geom\_point()

Genera puntos sobre un gráfico.

En la actividad de portafolios se utilizó para destacar el portafolio tangente.

---

## Histograma

Gráfico que representa cómo se distribuyen las observaciones de una variable.

---

## **geom\_histogram()**

Genera un histograma.

---

## **theme\_minimal()**

Aplica un estilo visual simplificado a un gráfico de ggplot2.

---

# Aleatoriedad y reproducibilidad

## `rnorm()`

Genera números pseudoaleatorios provenientes de una distribución normal.

---

## `rtriangle()`

Genera valores provenientes de una distribución triangular.

---

## `sample()`

Selecciona elementos aleatoriamente y permite especificar probabilidades.

---

## `set.seed()`

Fija la semilla del generador pseudoaleatorio.

```
set.seed(123)
```

Esto permite volver a ejecutar un análisis y obtener la misma secuencia aleatoria.

---

## Reproducibilidad

Capacidad de volver a ejecutar un análisis obteniendo resultados consistentes a partir del mismo:

```
Código  
+ Datos  
+ Dependencias  
+ Configuración
```

Herramientas utilizadas durante el curso relacionadas con reproducibilidad incluyen:

```
set.seed()  
Git  
GitHub  
renv  
R Markdown
```

---

# Git, GitHub y control de versiones

## Git

Sistema de control de versiones utilizado para registrar cambios realizados en archivos y código.

---

## GitHub

Plataforma que permite alojar repositorios Git y colaborar en proyectos.

---

## Repositorio

Carpeta de proyecto administrada mediante Git.

El repositorio del curso es:

```
https://github.com/sebaegana/clase_finanzas_r_202609
```

---

## `git clone`

Descarga una copia de un repositorio.

```
git clone https://github.com/sebaegana/clase_finanzas_r_202609.git
```

---

## Datos privados y GitHub

Los repositorios pueden configurarse como privados, pero esto no reemplaza las políticas de protección de datos.

No deberían almacenarse directamente en un repositorio:

```
Contraseñas  
Tokens
```

API Keys  
Credenciales  
Datos personales sensibles  
Información institucional confidencial

---

# Gestión del entorno

## **renv**

Herramienta utilizada para registrar las versiones de los paquetes utilizados por un proyecto de R.  
Ayuda a reproducir un entorno de trabajo en otro computador.

---

# Markdown y documentos reproducibles

## Markdown

Lenguaje de marcado ligero para estructurar documentos de texto.

```
# Título

## Subtítulo

**Negrita**

- Elemento
```

---

## R Markdown

Sistema que permite combinar en un mismo documento:

```
Texto
+ Código R
+ Resultados
+ Tablas
+ Gráficos
```

Puede instalarse mediante:

```
install.packages("rmarkdown")
```

---

# Inteligencia artificial y desarrollo asistido

## Desarrollo asistido por IA

Uso de modelos de inteligencia artificial para apoyar tareas de programación.

Durante las clases se utilizó como herramienta para reducir la fricción al construir scripts y trabajar con bibliotecas como `tidyverse` y `ggplot2`.

---

## Supervisión humana

Aunque una herramienta de IA puede generar código, una persona debe validar:

```
Supuestos financieros
Fórmulas
Unidades
Datos
Resultados
Interpretación
```

Generar código rápidamente no reemplaza la comprensión del problema.

---

# Preguntas frecuentes

## ¿Necesito utilizar Positron y RStudio simultáneamente?

No. Son entornos de desarrollo independientes y es posible trabajar con R utilizando cualquiera de ellos.

---

## ¿Necesito tener Python instalado para trabajar en Finanzas en R?

Para los ejercicios desarrollados exclusivamente en R, no.

Python apareció también durante las clases como herramienta complementaria.

---

## ¿Necesito saber Python para seguir el curso?

No necesariamente. El lenguaje principal utilizado para desarrollar los ejercicios financieros es R.

---

## ¿Por qué utilizamos 252 días para representar un año?

Porque corresponde a una aproximación habitual al número de días de negociación existentes durante un año.

---

## ¿Por qué utilizamos `set.seed()`?

Porque permite reproducir los mismos números pseudoaleatorios.

Esto facilita comparar resultados y detectar errores.

---

## ¿Qué diferencia existe entre retorno diario y retorno acumulado?

El retorno diario representa el cambio correspondiente a un período.

El retorno acumulado considera el efecto compuesto de todos los períodos anteriores.

---

## ¿Por qué no podemos simplemente sumar retornos?

Porque los retornos porcentuales se aplican sobre valores que cambian en cada período.

Por eso utilizamos:

```
cumprod(1 + retorno) - 1
```

---

## ¿Qué representa una matriz de correlación?

Permite estudiar la relación lineal entre los retornos de distintos activos.

---

## ¿Qué diferencia existe entre correlación y covarianza?

Ambas describen cómo dos variables se mueven conjuntamente.

La **covarianza** conserva las unidades y escalas de las variables.

La **correlación** estandariza esa relación entre -1 y 1.

En la optimización de Markowitz utilizamos directamente la matriz de covarianzas.

---

## ¿Por qué la covarianza es importante para un portafolio?

Porque el riesgo del portafolio depende tanto de la volatilidad individual de sus activos como de la forma en que estos se mueven conjuntamente.

Dos activos muy volátiles pueden, bajo determinadas relaciones entre sus retornos, producir un portafolio cuyo riesgo conjunto sea diferente del riesgo de cada activo considerado aisladamente.

---

## ¿Qué es la frontera eficiente?

Es el conjunto de portafolios eficientes que se obtiene al buscar, para distintos niveles de retorno, combinaciones con el menor riesgo permitido por el modelo y sus restricciones.

---

## ¿Cada punto de la frontera eficiente representa un portafolio diferente?

Sí. Cada punto puede contener diferentes:

```
Pesos
Retorno esperado
Volatilidad
Ratio de Sharpe
```

---

## ¿Qué significa que los pesos sumen 1?

Significa que se distribuye el 100 % del capital disponible.

```
w1 + w2 + ... + wn = 1
```

---

## ¿Qué significa $w \geq 0$ ?

Que los pesos no pueden ser negativos.

En el ejercicio esto implica que no se permiten posiciones cortas.

---

## ¿Qué hace quadprog?

Es un paquete de R utilizado para resolver problemas de programación cuadrática.

En clase se utilizó:

```
solve.QP()
```

para encontrar los pesos del portafolio bajo determinadas restricciones.

---

## ¿Por qué probamos muchos retornos objetivo?

Porque para construir la frontera eficiente necesitamos calcular múltiples portafolios.

En el ejercicio se utilizaron 50 retornos objetivo.

Cada retorno objetivo genera un nuevo problema de optimización.

---

## ¿Por qué el retorno objetivo se divide por 12?

En el ejercicio los niveles objetivo se expresan inicialmente en términos anuales y se dividen por 12 para trabajar en una escala mensual consistente con los datos utilizados en esa actividad.

---

## ¿Por qué la volatilidad se multiplica por $\sqrt{12}$ ?

Cuando se parte de una desviación estándar mensual, una forma habitual de anualizarla bajo los supuestos utilizados es:

```
Volatilidad anual =  
Volatilidad mensual  $\times \sqrt{12}$ 
```

---

## ¿Por qué el retorno se multiplica por 12 pero la volatilidad por $\sqrt{12}$ ?

Porque retorno esperado y volatilidad tienen propiedades matemáticas distintas.

En la aproximación utilizada en el ejercicio:

```
Retorno:  
mensual  $\times 12$   
  
Volatilidad:  
mensual  $\times \sqrt{12}$ 
```

---

## ¿Qué representa la tasa libre de riesgo?

Es una tasa utilizada como referencia para comparar el retorno de un activo o portafolio riesgoso.

En el ejercicio:

```
0.02
```

corresponde a una tasa anual del 2%.

---

## ¿Qué es el Ratio de Sharpe?

Relaciona el retorno adicional obtenido sobre la tasa libre de riesgo con la volatilidad asumida.

Simplificadamente:

```
Sharpe =  
Retorno excedente / Volatilidad
```

---

## ¿Un Sharpe mayor es siempre mejor?

Dentro de este criterio y comparando alternativas construidas de forma consistente, un Sharpe mayor representa más retorno excedente por unidad de volatilidad.

Sin embargo, sigue siendo un indicador basado en determinados supuestos y no resume por sí solo todas las dimensiones posibles del riesgo financiero.

---

## ¿Qué es el portafolio tangente?

Es el portafolio de la frontera eficiente que presenta el mayor ratio de Sharpe respecto de la tasa libre de riesgo utilizada.

En el ejercicio se obtiene con:

```
which.max(  
  frontera_markowitz$sharpe  
)
```

---

## ¿Qué es la Capital Allocation Line?

Es una línea que representa combinaciones entre el activo libre de riesgo y un portafolio riesgoso.

En la actividad se representa junto con la frontera eficiente y el portafolio tangente.

---

## ¿Por qué `solve.QP()` puede fallar para algún retorno objetivo?

Algunos retornos objetivo pueden no ser alcanzables dadas las restricciones utilizadas.

Por ejemplo, exigir simultáneamente:

```
Pesos que sumen 100%  
Sin posiciones negativas  
Un determinado retorno objetivo
```

puede producir un problema sin una solución válida para algunas combinaciones.

---

## ¿Para qué utilizamos `tryCatch()`?

Para evitar que una iteración fallida detenga todo el programa.

El programa puede registrar una advertencia y continuar con los demás retornos objetivo.

---

## ¿Qué diferencia existe entre `write.csv()` y `write_csv()`?

`write.csv()` forma parte de R base.

`write_csv()` pertenece al paquete `readr`, utilizado por `tidyverse`.

---

## ¿Para qué sirve una distribución triangular?

Puede utilizarse cuando conocemos aproximadamente:

```
Un mínimo
Un máximo
Un valor más probable
```

pero no contamos necesariamente con suficiente información histórica para estimar una distribución más compleja.

---

## ¿Por qué ejecutar una simulación 10.000 veces?

Porque permite observar una distribución de resultados posibles en lugar de depender de un único escenario.

---

## ¿Qué diferencia existe entre `ggplot2`, `matplotlib`, `seaborn` y `plotly`?

`ggplot2` pertenece principalmente al ecosistema R.

`matplotlib` y `seaborn` pertenecen principalmente al ecosistema Python.

`plotly` permite generar visualizaciones interactivas y posee implementaciones para diferentes lenguajes.

---

## ¿Es seguro subir información privada a GitHub?

Que un repositorio sea privado no significa que cualquier información deba almacenarse allí.

No deberían incorporarse directamente credenciales, contraseñas, tokens ni información sensible sin las medidas de seguridad correspondientes.

---

## ¿Para qué sirve renv?

Permite registrar y reconstruir las versiones de los paquetes utilizadas por un proyecto de R.

---

## ¿Qué diferencia existe entre Markdown y R Markdown?

Markdown permite estructurar documentos mediante texto.

R Markdown permite incorporar además código R y sus resultados.

---

## ¿La IA puede generar todo el código del análisis?

Puede ayudar considerablemente, pero el código debe ser validado.

Especialmente deben revisarse:

```
Supuestos
Unidades
Frecuencias
Fórmulas
Restricciones
Resultados
Interpretación financiera
```

---

# Enlaces y recursos utilizados en clases

## Material del curso

### Sitio Finanzas en R

[https://sebaegana.github.io/finanzas\\_en\\_r\\_webpage/clases\\_ia/](https://sebaegana.github.io/finanzas_en_r_webpage/clases_ia/)

Material general utilizado durante las sesiones.

### Notebook 01

[https://sebaegana.github.io/finanzas\\_en\\_r\\_webpage/clases\\_ia/notebook\\_01.html](https://sebaegana.github.io/finanzas_en_r_webpage/clases_ia/notebook_01.html)

Notebook utilizado como material de apoyo.

### Actividad 02 — Portafolios

[https://sebaegana.github.io/finanzas\\_en\\_r\\_webpage/clases\\_ia/actividad\\_02\\_portafolios.html](https://sebaegana.github.io/finanzas_en_r_webpage/clases_ia/actividad_02_portafolios.html)

Actividad utilizada para trabajar optimización de portafolios, frontera eficiente y teoría de Markowitz.

### Repositorio GitHub del curso

[https://github.com/sebaegana/clase\\_finanzas\\_r\\_202609](https://github.com/sebaegana/clase_finanzas_r_202609)

Para clonarlo:

```
git clone https://github.com/sebaegana/clase_finanzas_r_202609.git
```

---

## Aprender R y tidyverse

### R for Data Science

<https://r4ds.hadley.nz>

Libro de referencia para aprender análisis de datos con R.

### R for Data Science — español

<https://es.r4ds.hadley.nz>

Versión en español.

## **Paquetes tidyverse**

<https://tidyverse.org/packages/>

Información sobre los principales paquetes del ecosistema tidyverse.

## **Posit — Data Transformation Cheat Sheet**

<https://opensource.posit.co/resources/cheatsheets/data-transformation/>

Hoja de referencia para transformación de datos.

---

## **Reproducibilidad**

### **renv**

<https://rstudio.github.io/renv/articles/renv.html>

Documentación sobre gestión reproducible de dependencias.

---

# Mapa general de conceptos del curso

Los contenidos trabajados hasta el momento pueden entenderse como una progresión:

```
R y herramientas
  ↓
Manipulación de datos
  ↓
Retornos financieros
  ↓
Estadística y volatilidad
  ↓
Correlación y covarianza
  ↓
Simulación
  ↓
Riesgo y retorno
  ↓
Portafolios
  ↓
Optimización Media-Varianza
  ↓
Frontera eficiente
  ↓
Ratio de Sharpe
  ↓
Portafolio tangente
```

A nivel tecnológico, el flujo de trabajo incorpora:

```
R
+ tidyverse
+ ggplot2
+ Git / GitHub
+ renv
+ Markdown / R Markdown
+ herramientas de IA
```

El objetivo del curso no es solamente aprender sintaxis, sino utilizar estas herramientas para **construir análisis financieros reproducibles, entender el vínculo entre riesgo y retorno, automatizar procesos analíticos y mantener la capacidad de interpretar y validar los**

resultados obtenidos.